LAPORAN PRAKTIKUM

JOBSHEET 7 - PEMROGRAMAN PHP DASAR DAN FORM HANDLING



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Modularisasi Antarmuka dengan Header dan Footer 3

2.1.1. 1.1 Penerapan require_once dan konstanta `__DIR__` 3

2.1.2. Langkah-langkah 4

2.1.3. Pertanyaan 5

2.2. Percobaan 2: Pemrosesan Form Tambah Data dan Validasi Sisi Server 6

2.2.1. 2.1 Sanitasi Input dan Validasi Berlapis 6

2.2.2. 2.2 Penolakan Data Tidak Valid 7

2.2.3. Langkah-langkah 7

2.2.4. Pertanyaan 8

2.3. Percobaan 3: Penyimpanan Data pada Session dan Arsitektur PRG 9

2.3.1. 3.1 Pola Navigasi Post/Redirect/Get (PRG) 9

2.3.2. 3.2 Notifikasi Flash Sekali-Pakai (Flash Message) 9

2.3.3. Langkah-langkah 10

2.3.4. Pertanyaan 12

3. Tugas 13

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 7 13

1. Tujuan

- Memahami konsep dasar arsitektur Server-Side Rendering (SSR) menggunakan bahasa pemrograman PHP.
- Menguasai teknik modularisasi antarmuka web mematuhi prinsip DRY (Don't Repeat Yourself) melalui direktif require_once dan konstanta absolut `__DIR__`.
- Menerapkan penanganan status (state management) pada protokol stateless HTTP memanfaatkan variabel superglobal `$_SESSION`.
- Mengimplementasikan pemrosesan data form masukan melalui metode HTTP POST dan sanitasi nilai masukan.
- Membangun lapisan validasi data sisi server (Server-Side Validation) yang tangguh dan tidak dapat dimanipulasi oleh peramban klien.
- Menerapkan arsitektur navigasi Post/Redirect/Get (PRG) menggunakan fungsi header('Location: ...') dan terminasi eksekusi exit.
- Merancang sistem notifikasi sekali-pakai (Flash Messages) berbasis session dengan mekanisme pembersihan variabel unset().
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/DnPWeb2026.

2. Praktikum

2.1. Percobaan 1: Modularisasi Antarmuka dengan Header dan Footer

Pada percobaan pertama, struktur dokumen aplikasi SIMPUS-Mini dimigrasikan dari file statis HTML ke skrip PHP dinamis. Kerangka tata letak yang berulang seperti navigasi `<header>` dan penutup `<footer>` dipisahkan ke dalam folder `includes/`.

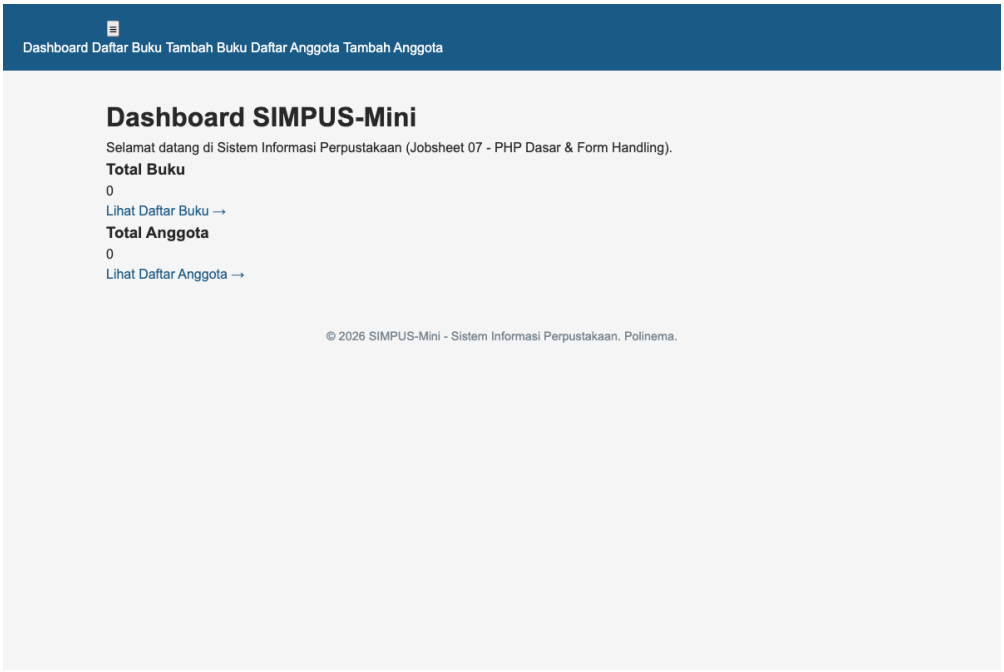
2.1.1. 1.1 Penerapan require_once dan konstanta `__DIR__`

Modularisasi dilakukan menggunakan fungsi bawaan `require_once` yang dipadukan dengan konstanta direktori absolut `__DIR__`:

```
<?php
$page_title = "Dashboard";
require_once __DIR__ . '/includes/header.php';
?>
```

Penggunaan `__DIR__` menjamin keakuratan resolusi path berkas di lingkungan server tanpa terpengaruh oleh direktori kerja peramban. Di dalam `header.php`,

sesi diinisialisasi secara terpusat melalui `session_start()`, dan komponen navigasi utama disajikan seragam untuk seluruh halaman.



Gambar 1.1: Antarmuka Dashboard SIMPUS-Mini Menampilkan Statistik Data Buku dan Anggota dari Sesi.

2.1.2. Langkah-langkah

1. Membuat berkas `includes/header.php` yang memuat pembukaan tag HTML, inisialisasi session_start(), serta elemen header navbar.
2. Membuat berkas `includes/footer.php` yang memuat penutup tag dokumen HTML.
3. Mengintegrasikan kedua berkas pembungkus pada `index.php` menggunakan direktif `require_once`.
4. Memverifikasi tata letak antarmuka di peramban web pada alamat `http://localhost:8000/jobsheet07/src/index.php`.

2.1.3. Pertanyaan

1. Mengapa penggunaan `require_once` lebih aman dibandingkan `include` pada pemuatan berkas inti?
    - `require_once` menghasilkan kesalahan fatal (**Fatal Error: E_COMPILE_ERROR**) dan menghentikan seluruh eksekusi program jika berkas yang ditargetkan tidak ditemukan, mencegah berjalannya skrip dalam kondisi dependensi yang rusak. Selain itu, imbuhan `_once` memastikan berkas tersebut hanya dimuat tepat satu kali, mencegah terjadinya deklarasi ulang fungsi atau variabel ganda (**Cannot redeclare function**).

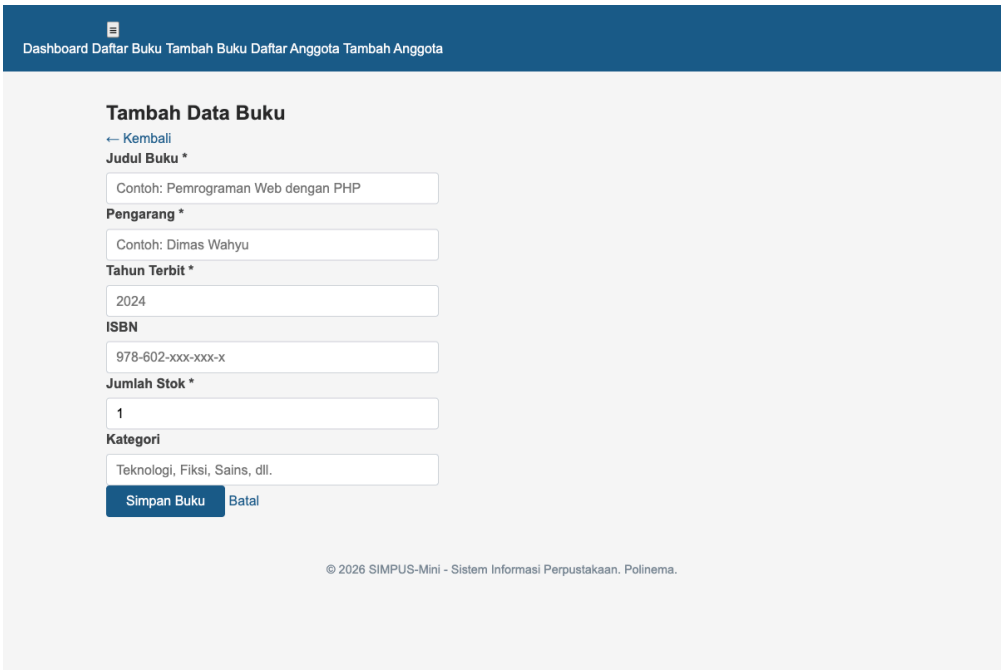
2.2. Percobaan 2: Pemrosesan Form Tambah Data dan Validasi Sisi Server

Percobaan kedua mengimplementasikan alur penerimaan data formulir tambah buku dan anggota melalui metode HTTP POST pada skrip pengendali `proses_tambah.php`.

2.2.1. 2.1 Sanitasi Input dan Validasi Berlapis

Data masukan pengguna diekstrak dari variabel superglobal `$_POST` dan disanitasi menggunakan fungsi `trim()`. Berbeda dengan Jobsheet 5 yang mengandalkan JavaScript di peramban, validasi di Jobsheet 7 dieksekusi murni di sisi server:

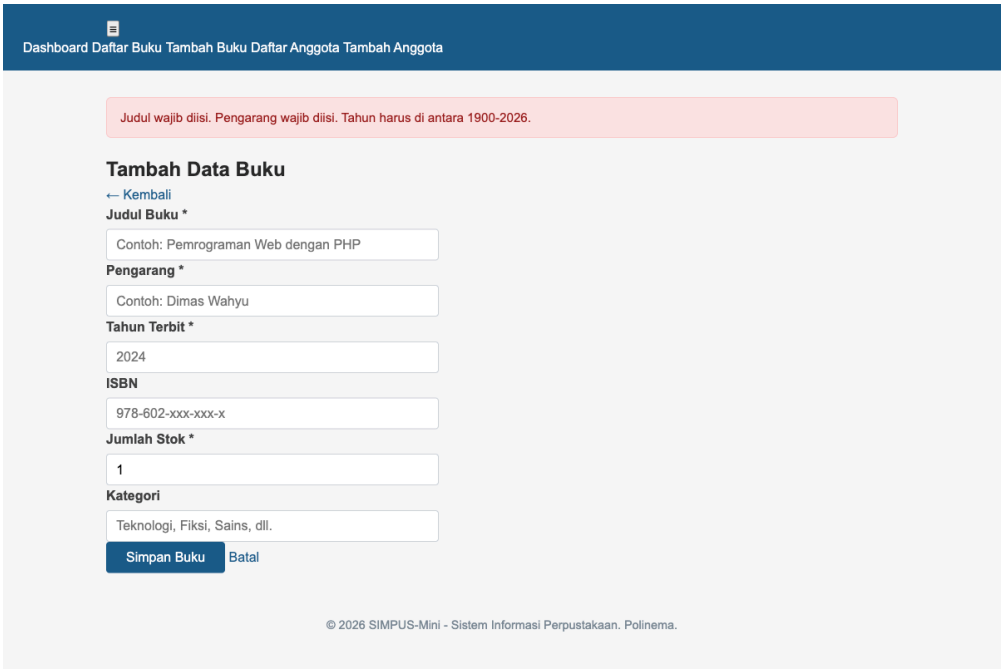
```
$errors = [];
if ($judul === '') {
    $errors[] = "Judul wajib diisi.";
}
if ($pengarang === '') {
    $errors[] = "Pengarang wajib diisi.";
}
if (!is_numeric($tahun) || $tahun < 1900 || $tahun > 2026) {
    $errors[] = "Tahun harus di antara 1900-2026.";
}
if (!is_numeric($stok) || $stok < 0) {
    $errors[] = "Stok tidak boleh negatif.";
}
```



Gambar 1.2: Tampilan Form Tambah Data Buku dengan Berbagai Bidang Isian Terstruktur.

2.2.2. 2.2 Penolakan Data Tidak Valid

Apabila array `$errors` tidak kosong, alur eksekusi langsung membatalkan penyimpanan data, mencatat pesan kesalahan ke dalam variabel flash session, dan membelokkan kembali pengguna ke form input:



Gambar 1.3: Notifikasi Galat Server-Side Berwarna Merah Muncul Saat Bidang Form Wajib Dikosongkan.

2.2.3. Langkah-langkah

1. Menyusun form HTML pada `books/tambah.php` dengan atribut `action="proses_tambah.php" method="POST"`.
2. Mengembangkan skrip pemrosesan `books/proses_tambah.php` yang mengekstrak nilai `$_POST`.
3. Menulis aturan validasi kelayakan data (keberadaan teks, rentang tahun, dan batas minimal stok).
4. Menguji pengiriman formulir dengan data tidak valid dan memverifikasi penolakan penyimpanan di sisi server.

2.2.4. Pertanyaan

1. Mengapa validasi sisi server mutlak diperlukan meskipun validasi client-side sudah ada?

    - Validasi sisi klien dapat dilewati dengan sangat mudah oleh pengguna melalui penonaktifan JavaScript di peramban atau pengiriman request langsung menggunakan cURL dan Postman. Validasi sisi server merupakan satu-satunya lapisan pertahanan yang berada 100% di bawah kendali pengembang dan tidak dapat dimanipulasi oleh klien.

2.3. Percobaan 3: Penyimpanan Data pada Session dan Arsitektur PRG

Percobaan ketiga mengimplementasikan persistensi data sementara pada variabel superglobal `$_SESSION['buku']` dan menerapkan pola navigasi Post/Redirect/Get (PRG).

2.3.1. 3.1 Pola Navigasi Post/Redirect/Get (PRG)

Setelah data buku yang valid ditambahkan ke array sesi, server tidak langsung merender halaman HTML, melainkan mengirimkan header pengalihan lokasi:

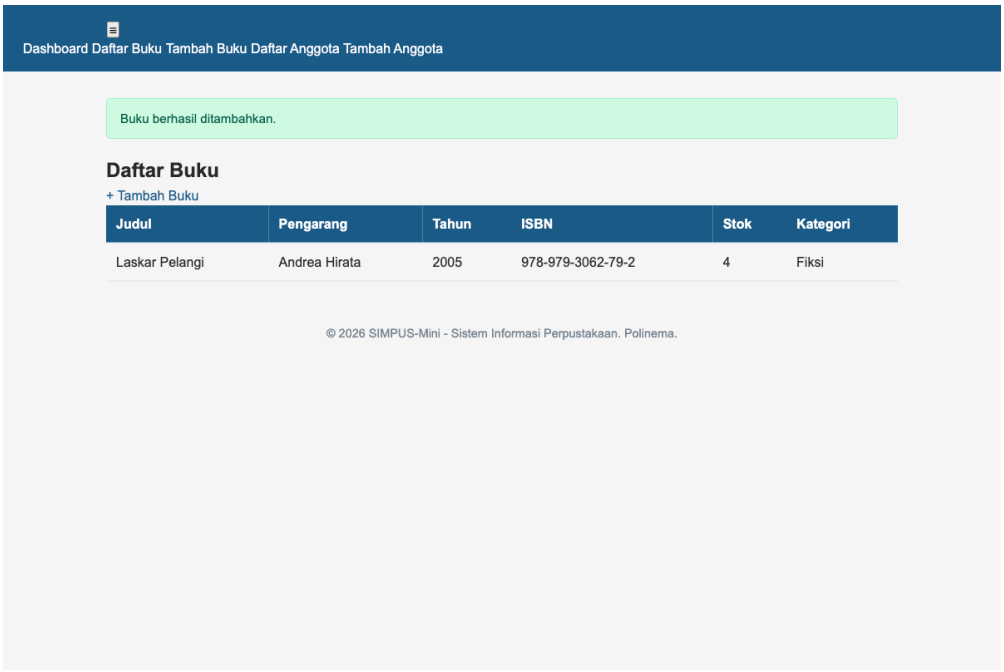
```
$_SESSION['buku'][] = [
    'judul' => $judul,
    'pengarang' => $pengarang,
    'tahun' => (int) $tahun,
    'isbn' => $isbn,
    'stok' => (int) $stok,
    'kategori' => $kategori,
];

$_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku berhasil ditambahkan.'];
header('Location: list.php');
exit;
```

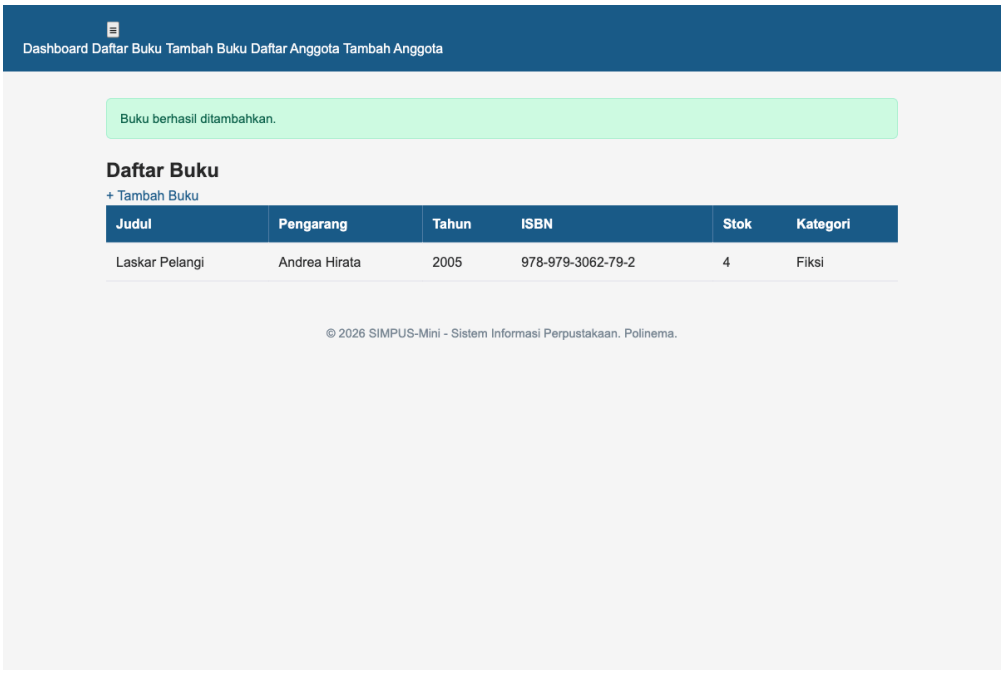
Panggilan `exit;` wajib disertakan tepat setelah `header('Location: ...')` untuk menghentikan pemrosesan skrip PHP lebih lanjut di latar belakang.

2.3.2. 3.2 Notifikasi Flash Sekali-Pakai (Flash Message)

Pada saat halaman tujuan `list.php` dibuka, komponen `includes/header.php` memeriksa keberadaan `$_SESSION['flash']`. Pesan notifikasi sukses berwarna hijau ditampilkan, dan variabel tersebut langsung dihapus dari memori server melalui perintah `unset($_SESSION['flash'])`:



Gambar 1.4: Notifikasi Flash Berhasil Ditampilkan Tepat Satu Kali Pasca-Pengalihan HTTP.



Gambar 1.5: Data Buku yang Baru Ditambahkan Berhasil Direnderdari Variabel `$_SESSION['buku']`.

2.3.3. Langkah-langkah

1. Menyimpan array asosiatif buku baru ke dalam indeks larik `$_SESSION['buku'][]`.

2. Mengirim header pengalihan `header('Location: list.php')` diikuti perintah terminasi `exit`.
3. Mengembangkan komponen perender flash message di dalam `includes/header.php` lengkap dengan pembersihan `unset()`.
4. Membuka halaman `list.php` untuk memverifikasi data baru serta memastikan flash message lenyap saat halaman di-refresh.

2.3.4. Pertanyaan

1. Apa konsekuensi fatal jika perintah `exit;` diabaikan setelah pemanggilan `header('Location: ...')`?

    - Tanpa perintah `exit;`, mesin PHP akan tetap melanjutkan eksekusi seluruh baris kode yang berada di bawahnya hingga berkas selesai, meskipun peramban telah diinstruksikan untuk berpindah halaman. Hal ini berpotensi memicu eksekusi logika yang tidak diinginkan, konsumsi sumber daya komputasi sia-sia, atau kebocoran data sensitif yang tercetak di respons stream.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 7

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 7:

No	Pertanyaan dan Jawaban Konseptual
1	**Soal:** Jelaskan perbedaan mendasar antara eksekusi kode Server-Side (PHP) dan Client-Side (HTML/JS)! Mengapa kode sumber PHP tidak pernah terlihat saat pengguna melakukan “View Page Source” di peramban? **Jawaban:** Kode **Server-Side** seperti PHP dieksekusi secara tuntas di lingkungan komputer server web sebelum paket data respons dikirimkan melalui jaringan internet. Mesin interpreter PHP mengolah seluruh logika variabel, pengkondisian, dan manipulasi sesi, kemudian memancarkan hasil akhirnya dalam format dokumen HTML murni. Klien peramban web hanya bertindak sebagai penerima dokumen HTML tersebut. Sebaliknya, kode **Client-Side** (HTML/CSS/JS) dikirimkan dalam bentuk teks mentah dan diparsing di mesin peramban pengguna. Karena seluruh skrip PHP telah selesai diproses di server dan hanya menyisakan keluaran HTML murni, pengguna yang membuka fitur **View Page Source** tidak akan pernah melihat sebaris pun kode logika PHP aslinya.
2	**Soal:** Apa fungsi konstanta `__DIR__` saat menyertakan berkas menggunakan `require_once __DIR__ . '/../includes/header.php'`, dan mengapa teknik modularisasi ini mematuhi prinsip DRY (Don’t Repeat Yourself)? **Jawaban:** Konstanta bawaan `__DIR__` mengembalikan path direktori absolut dari sistem berkas tempat skrip yang sedang aktif berada. Penerapan `__DIR__` menghilangkan ambiguitas penelusuran path yang kerap terjadi jika hanya mengandalkan path relatif biasa saat skrip dipanggil

No	Pertanyaan dan Jawaban Konseptual
	dari working directory yang bervariasi. Modularisasi ini mematuhi prinsip DRY karena blok elemen navigasi `<header>`, pemanggilan lembar gaya CSS, dan logika inisialisasi sesi cukup didefinisikan satu kali di berkas terpusat `includes/header.php`. Setiap pembaruan menu atau modifikasi aset global di masa depan cukup dilakukan pada satu berkas tersebut tanpa perlu menyalin ulang kode ke puluhan halaman lain.
3	**Soal:** Bagaimana mekanisme `$_SESSION` menjembatani data antar-permintaan HTTP yang bersifat stateless? Mengapa data pada `$_SESSION` di Jobsheet 7 bersifat sementara (ephemeral)? **Jawaban:** Protokol transfer HTTP beroperasi tanpa status (**stateless**), yang berarti setiap permintaan dianggap independen tanpa memori terhadap request sebelumnya. PHP menjembatani keterbatasan ini melalui mekanisme Session: fungsi `session_start()` menerbitkan identifier sesi unik yang ditransmisikan ke peramban klien melalui cookie HTTP bernama `PHPSESSID`. Pada setiap request selanjutnya, peramban secara otomatis melampirkan cookie tersebut sehingga server dapat mengaitkannya kembali dengan ruang penyimpanan memori atau berkas sesi terkait di server melalui variabel superglobal `$_SESSION`. Data pada Jobsheet 7 bersifat sementara (**ephemeral**) karena disimpan di dalam media penyimpanan sementara server yang masa berlakunya terikat pada siklus hidup sesi peramban. Jika sesi kadaluarsa, peramban ditutup sepenuhnya, atau layanan web server dihentikan, data tersebut akan musnah permanen.
4	**Soal:** Mengapa validasi sisi server (Server-Side Validation) pada `proses_tambah.php` wajib diimplementasikan meskipun form input sudah memiliki atribut required dan validasi JavaScript di sisi klien? **Jawaban:** Validasi sisi klien beroperasi di lingkungan eksekusi peramban

No	Pertanyaan dan Jawaban Konseptual
	pengguna yang sepenuhnya berada di luar kendali pengembang (**untrusted execution environment**). Pengguna tingkat lanjut dapat dengan mudah mematikan fitur JavaScript melalui konfigurasi browser, mencabut atribut `required` lewat DevTools Elements Inspector, atau langsung mengirimkan payload HTTP POST berbahaya menggunakan peralatan otomasi seperti cURL dan Postman. Oleh karena itu, validasi sisi server merupakan benteng integritas data yang wajib dan mutlak karena dijalankan di lingkungan server tepercaya yang tidak dapat dimanipulasi oleh klien.
5	**Soal:** Jelaskan alur kerja pola Post/Redirect/Get (PRG) dan bagaimana mekanisme Flash Message (`$_SESSION['flash']` dan `unset()`) mencegah duplikasi data saat pengguna me-refresh halaman! **Jawaban:** Pola arsitektur PRG memisahkan tugas mutasi data dari presentasi hasil akhir: 1. **POST**: Formulir mentransmisikan data ke skrip pemroses (`proses_tambah.php`). 2. **Redirect**: Setelah data divalidasi dan disimpan, server merespons dengan header HTTP 302 `header('Location: list.php')` diikuti `exit;`. 3. **GET**: Peramban klien mengeksekusi permintaan HTTP GET baru ke URL target `list.php`. Pemisahan ini memastikan bahwa jika pengguna menekan tombol refresh (F5), peramban hanya me-refresh request GET aman dan tidak memicu pengiriman ulang formulir (**form resubmission**). Mekanisme **Flash Message** menampung pesan status pada variabel `$_SESSION['flash']`. Ketika halaman tujuan me-render pesan tersebut, eksekusi fungsi `unset($_SESSION['flash'])` langsung memusnahkannya dari memori sesi sehingga notifikasi hanya tampil tepat satu kali dan tidak muncul kembali pada kunjungan halaman berikutnya.

LAPORAN PRAKTIKUM
JOBSHEET 8 - INTEGRASI BASIS DATA POSTGRESQL DAN PDO



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Perancangan Skema Database dan Migrasi DDL 3

2.1.1. 1.1 Struktur DDL Tabel Buku dan Anggota 3

2.1.2. Langkah-langkah 4

2.1.3. Pertanyaan 6

2.2. Percobaan 2: Pembuatan Modul Koneksi PDO (PHP Data Objects) 7

2.2.1. 2.1 Konfigurasi DSN dan Error Handling Berbasis Exception 7

2.2.2. Langkah-langkah 7

2.2.3. Pertanyaan 9

2.3. Percobaan 3: Penyisipan dan Pembacaan Data Menggunakan Prepared Statements 10

2.3.1. 3.1 Eliminasi Kerentanan SQL Injection dengan Prepared Statements 10

2.3.2. 3.2 Pembacaan Data dan Perenderan Tabel 10

2.3.3. Langkah-langkah 12

2.3.4. Pertanyaan 13

3. Tugas 14

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 8 14

1. Tujuan

- Memahami konsep dasar persistensi data menggunakan sistem manajemen basis data relasional (RDBMS) PostgreSQL.
- Menguasai teknik Data Definition Language (DDL) untuk perancangan skema tabel buku dan anggota pada berkas 01_buku_anggota.sql.
- Mengkonfigurasi koneksi database modern berbasis pustaka PHP Data Objects (PDO) pada berkas includes/koneksi.php.
- Menerapkan konfigurasi penanganan error berbasis PDOException serta fetch mode array asosiatif (PDO::FETCH_ASSOC).
- Menerapkan teknik Prepared Statements berparameter untuk memitigasi kerentanan keamanan SQL Injection secara mutlak.
- Memigrasikan pembacaan dan penyimpanan data aplikasi SIMPUS-Mini dari $_SESSION ke tabel relasional PostgreSQL.
- Menganalisis penegakan integritas data pada tingkat database melalui batasan PRIMARY KEY dan UNIQUE constraints.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/DnPWeb2026.

2. Praktikum

2.1. Percobaan 1: Perancangan Skema Database dan Migrasi DDL

Pada percobaan pertama, struktur persistensi data aplikasi SIMPUS-Mini dimigrasikan dari memori sementara sesi ke basis data PostgreSQL bernama `simpus_mini`.

2.1.1. 1.1 Struktur DDL Tabel Buku dan Anggota

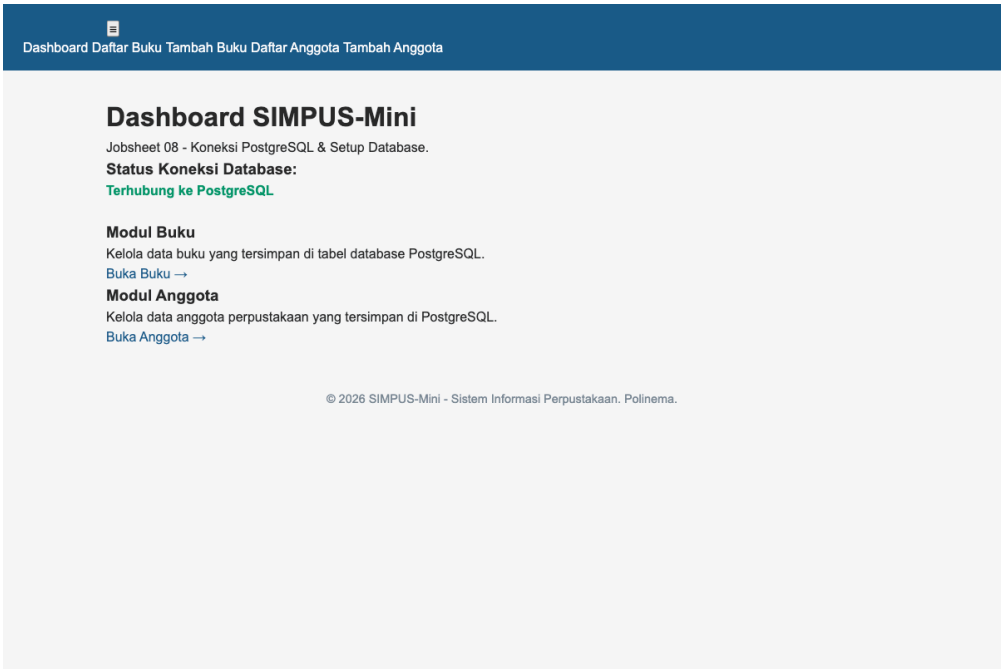
Skrip migrasi SQL `sql/01_buku_anggota.sql` dieksekusi untuk mengonstruksi dua tabel relasional utama:

```
CREATE TABLE IF NOT EXISTS buku (
    id SERIAL PRIMARY KEY,
    judul VARCHAR(255) NOT NULL,
    pengarang VARCHAR(255) NOT NULL,
    tahun INTEGER NOT NULL,
    isbn VARCHAR(50),
    stok INTEGER NOT NULL DEFAULT 0,
    kategori VARCHAR(50)
```

```
);

CREATE TABLE IF NOT EXISTS anggota (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(255) NOT NULL,
    no_anggota VARCHAR(50) NOT NULL UNIQUE,
    alamat VARCHAR(255),
    no_hp VARCHAR(30)
);
```

Penggunaan kolom `id SERIAL PRIMARY KEY` menjamin setiap baris memiliki kunci pengenal unik (**auto-increment**), sedangkan `no_anggota VARCHAR(50) NOT NULL UNIQUE` mencegah terjadinya nomor anggota ganda.



Gambar 1.1: Indikator Status Koneksi Database Berhasil Terhubung ke Layanan PostgreSQL Lokal.

2.1.2. Langkah-langkah

1. Membuat basis data `simpus_mini` menggunakan perintah CLI PostgreSQL.
2. Mengeksekusi skrip `sql/01_buku_anggota.sql` melalui command `psql -d simpus_mini -f sql/01_buku_anggota.sql`.
3. Memverifikasi struktur skema tabel melalui perintah periksa relasi `\dt` pada terminal psql.

4. Memeriksa status keterhubungan database pada antarmuka Dashboard `index.php`.

2.1.3. Pertanyaan

1. Mengapa migrasi dari `$_SESSION` ke PostgreSQL merupakan langkah arsitektural yang krusial?
    - Variabel `$_SESSION` bersifat sementara (**in-memory**) dan terisolasi per pengguna. Data akan musnah saat peramban ditutup atau server di-restart, serta tidak dapat diakses bersama antar-pengguna. PostgreSQL menyediakan persistensi data permanen, konkurensi multi-pengguna yang konsisten, serta integritas transaksi berbasis prinsip ACID.

2.2. Percobaan 2: Pembuatan Modul Koneksi PDO (PHP Data Objects)

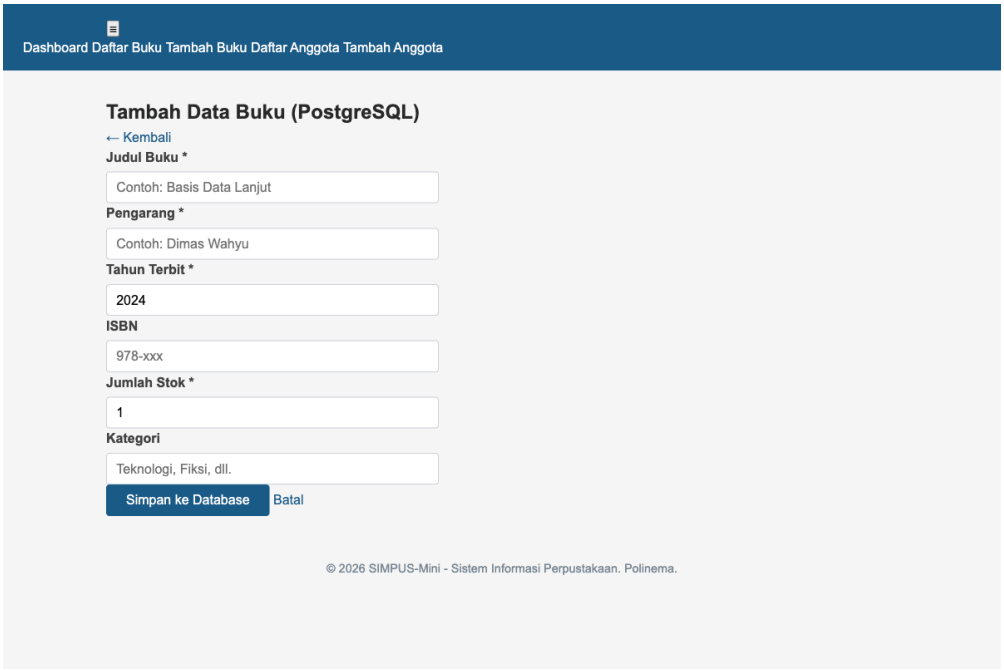
Percobaan kedua membangun modul konektivitas database terpusat pada berkas `includes/koneksi.php` menggunakan pustaka PDO.

2.2.1. 2.1 Konfigurasi DSN dan Error Handling Berbasis Exception

PDO diinisialisasi menggunakan string koneksi DSN (**Data Source Name**) PostgreSQL dan dikonfigurasi untuk melempar pengecualian `PDOException`:

```
$host = "localhost";
$port = "5432";
$db   = "simpus_mini";
$user = "postgres";
$pass = "postgres";

try {
    $pdo = new PDO("pgsql:host=$host;port=$port;dbname=$db", $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    $db_error = $e->getMessage();
}
```



Gambar 1.2: Formulir Tambah Data Buku yang Terintegrasi dengan Lapisan Penyimpanan PostgreSQL.

2.2.2. Langkah-langkah

1. Menyusun berkas modul `includes/koneksi.php` dengan parameter koneksi host, port, dbname, user, dan password.

2. Mengatur atribut penanganan galat `PDO::ATTR_ERRMODE` ke `PDO::ERRMODE_EXCEPTION`.
3. Mengatur default pengambilan data ke mode asosiatif `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`.
4. Menguji instansiasi objek PDO melalui blok try...catch.

2.2.3. Pertanyaan

1. Apa keuntungan menggunakan PDO dibandingkan ekstensi spesifik database seperti `pg_connect()`?

    - PDO menyediakan antarmuka akses data seragam yang portabel antar-berbagai DBMS (PostgreSQL, MySQL, SQLite) tanpa perlu mengubah logika pemanggilan query aplikasi jika terjadi pergantian engine database. Selain itu, PDO mendukung prepared statements berparameter native dan error handling berbasis exception yang tangguh.

2.3. Percobaan 3: Penyisipan dan Pembacaan Data Menggunakan Prepared Statements

Percobaan ketiga mengimplementasikan pembacaan data (`SELECT`) dan penyimpanan data (`INSERT`) ke database PostgreSQL menggunakan teknik **Prepared Statements**.

2.3.1. 3.1 Eliminasi Kerentanan SQL Injection dengan Prepared Statements

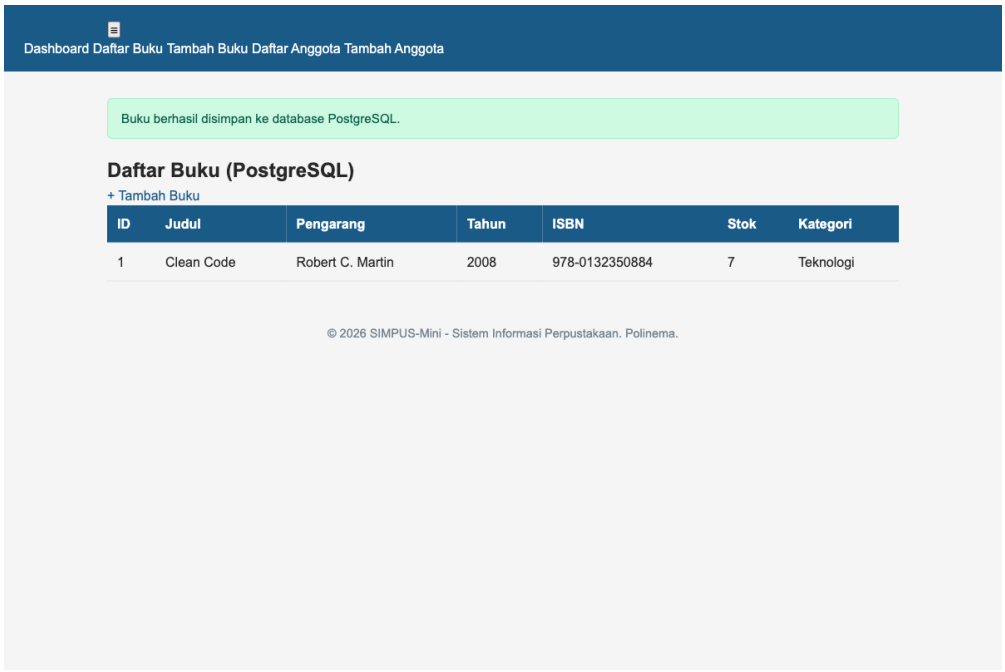
Penyimpanan data pada `books/proses_tambah.php` menggunakan query berparameter `:placeholder`:

```
$stmt = $pdo->prepare("INSERT INTO buku (judul, pengarang, tahun, isbn, stok,
kategori)
                       VALUES (:judul, :pengarang, :tahun, :isbn, :stok, :kategori)");
$stmt->execute([
    ':judul' => $judul,
    ':pengarang' => $pengarang,
    ':tahun' => (int) $tahun,
    ':isbn' => $isbn,
    ':stok' => (int) $stok,
    ':kategori' => $kategori,
]);
```

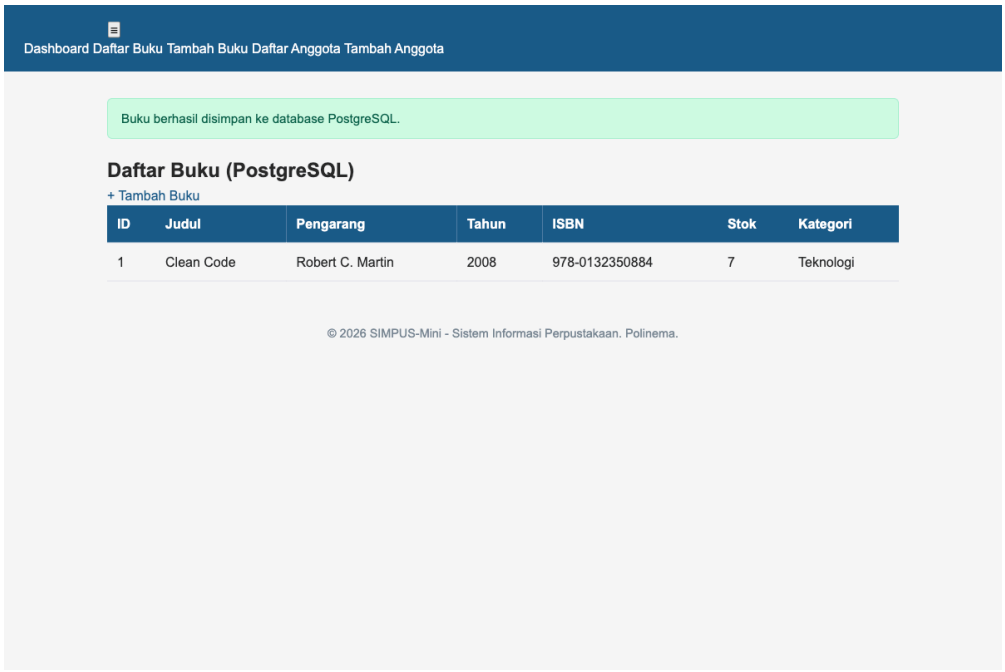
Dengan metode ini, struktur SQL dikompilasi terlebih dahulu di engine PostgreSQL, sedangkan nilai masukan dikirim secara terpisah sebagai data literal mentah. Hal ini mengeliminasi risiko serangan **SQL Injection** secara mutlak.

2.3.2. 3.2 Pembacaan Data dan Perenderan Tabel

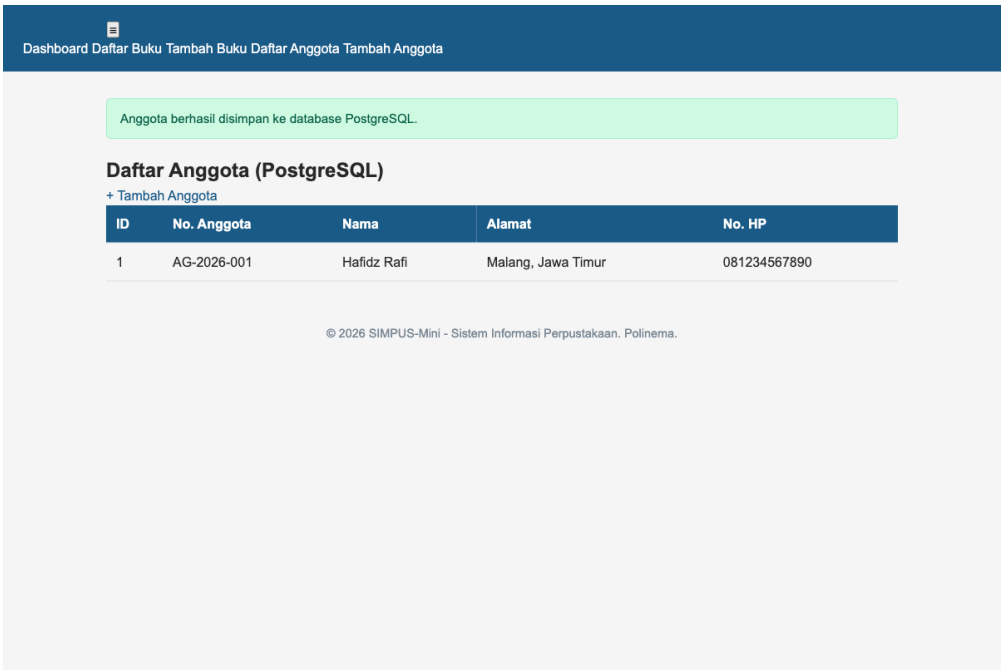
Pada `books/list.php`, data dibaca menggunakan `$stmt = $pdo->query("SELECT * FROM buku ORDER BY id DESC")` dan direnderer secara dinamis:



Gambar 1.3: Notifikasi Berhasil Menyimpan Data Buku ke Database PostgreSQL Pasca-Redirect HTTP.



Gambar 1.4: Tabel Daftar Buku Berhasil Menampilkan Rekaman Persisten dari Tabel PostgreSQL.



Gambar 1.5: Tabel Daftar Anggota Berhasil Memuat Data Anggota Persisten dari Database.

2.3.3. Langkah-langkah

1. Memodifikasi `proses_tambah.php` untuk mengeksekusi prepared statements INSERT dengan parameter binding.
2. Memodifikasi `list.php` untuk membaca data dari tabel database via `query()->fetchAll()`.
3. Menguji penambahan buku baru dan memeriksa keberadaan data di database PostgreSQL.
4. Menguji penambahan anggota baru dan memverifikasi perenderan data pada modul anggota.

2.3.4. Pertanyaan

1. Bagaimana mekanisme Prepared Statements menggagalkan manipulasi SQL berbahaya?
    - Prepared statements memisahkan sintaks instruksi SQL dari nilai data masukan. Kompilasi kerangka query telah selesai sebelum data diinjeksikan. Nilai masukan hanya diperlakukan murni sebagai string literal teks, sehingga karakter kutip atau operator SQL berbahaya (seperti `' OR 1=1 --`) tidak akan pernah ditafsirkan sebagai perintah oleh database parser.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 8

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 8:

No	Pertanyaan dan Jawaban Konseptual
1	**Soal:** Mengapa arsitektur penyimpanan data aplikasi SIMPUS-Mini harus dimigrasikan dari `$_SESSION` (pada Jobsheet 7) ke sistem manajemen basis data relasional seperti PostgreSQL (pada Jobsheet 8)? **Jawaban:** Variabel superglobal `$_SESSION` beroperasi di dalam memori sementara server yang terisolasi per masing-masing sesi pengguna. Data sesi tersebut akan langsung musnah ketika jendela peramban ditutup atau proses server web dimatikan, serta tidak memungkinkan kolaborasi data antar-pengguna yang berbeda. Migrasi ke PostgreSQL menyediakan persistensi data permanen di media penyimpanan sekunder, mendukung konkurensi transaksi multi-pengguna yang konsisten, memungkinkan pembentukan relasi data terstruktur, serta menjamin keandalan data transaksi sesuai kaidah ACID (**Atomicity, Consistency, Isolation, Durability**).
2	**Soal:** Apa fungsi dan keunggulan menggunakan PHP Data Objects (PDO) sebagai pustaka koneksi basis data dibandingkan menggunakan fungsi pustaka spesifik seperti `pg_connect()` atau `mysqli_connect()`? **Jawaban:** PDO bertindak sebagai lapisan abstraksi akses basis data (**Database Abstraction Layer**) terpadu di lingkungan PHP. Keunggulan utamanya mencakup: 1. **Portabilitas Kode**: Pengembang menggunakan antarmuka pemanggilan metode yang konsisten (`prepare()`, `execute()`, `fetch()`) untuk berkomunikasi dengan belasan jenis RDBMS

No	Pertanyaan dan Jawaban Konseptual
	berbeda. Jika di masa depan sistem bermigrasi ke DBMS lain, kode query aplikasi tidak perlu dirombak total. 2. **Keamanan Bawaan**: PDO mendukung **Prepared Statements** berparameter secara native untuk mitigasi SQL Injection. 3. **Manajemen Galat Modern**: Mendukung penanganan galat terstruktur berbasis pengecualian objek `PDOException`.
3	**Soal:** Jelaskan fungsi dari konfigurasi atribut `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` dan `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC` pada file `koneksi.php`! **Jawaban:** Konfigurasi `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` menginstruksikan PDO untuk melempar objek pengecualian `PDOException` secara otomatis setiap kali query mengalami kegagalan sintaksis atau koneksi terputus. Hal ini memungkinkan isolasi error yang rapi menggunakan blok `try...catch`. Sementara itu, `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC` mengatur agar hasil pembacaan baris rekaman data secara baku dikembalikan sebagai array asosiatif (nama kolom sebagai kunci array, misal `$buku['judul']`), mengeliminasi duplikasi data array numerik serta menghemat alokasi memori runtime.
4	**Soal:** Bagaimana teknik Prepared Statements (`$stmt->prepare()` dan `$stmt->execute()`) mengeliminasi risiko serangan SQL Injection secara mutlak? **Jawaban:** Pada penggabungan string query biasa, input pengguna disatukan langsung ke dalam teks perintah SQL sehingga karakter jahat dapat membelokkan logika perintah. Dengan Prepared Statements, siklus eksekusi dipisahkan menjadi dua tahapan ketat:

No	Pertanyaan dan Jawaban Konseptual
	1. **Fase Kompilasi (Prepare)**: Struktur perintah SQL dikirim dan dikompilasi terlebih dahulu oleh parser database dengan placeholder parameter (`:parameter`). 2. **Fase Pengikatan Data (Execute)**: Nilai masukan pengguna dikirim secara terpisah murni sebagai nilai data literal mentah. Database engine tidak akan pernah menafsirkan karakter kutip atau ekspresi logika tambahan sebagai perintah SQL, menggagalkan segala bentuk eksploitasi injeksi kode.
5	**Soal:** Jelaskan peran integritas data dari klausa `SERIAL PRIMARY KEY` pada tabel buku dan klausa `NOT NULL UNIQUE` pada kolom `no_anggota` di skema `01_buku_anggota.sql`! **Jawaban:** Klausa `SERIAL PRIMARY KEY` pada PostgreSQL secara otomatis mendefinisikan generator sequence angka urut (**auto-increment**) yang menghasilkan bilangan bulat unik bagi setiap baris rekaman baru, sekaligus menetapkannya sebagai kunci utama tak-boleh-kosong (**Primary Key**) yang menjadi pengenal unik entitas. Klausa `NOT NULL UNIQUE` pada `no_anggota` menegakkan integritas domain: `NOT NULL` mewajibkan setiap anggota memiliki nomor identitas keanggotaan terdaftar, sedangkan `UNIQUE` menginstruksikan database engine untuk menolak pencatatan nomor anggota duplikat, menjamin keunikan identitas anggota di seluruh sistem.